

BootIT

Ai Ting Lee

Based on slides by Niek Janssen

-- Based on slides by Sebastian Mateos Nicolajsen

---- Based on slides by Claus Brabrand

Welcome!!!

Teachers



Ai Ting Lee

- She/Her
- From Taiwan 🍹🍹🍹
- Mandarin (Native), English, Danish (Terrible)
- Full-stack developer at Dreamdata
ITU TA in Practical Concurrent and Parallel Programming
- MSc in Software Design (2024 - June 2026)
- Bachelor of Education
- Scuba diver



Amanda Lima Soares Da Cunha

- She/Her
- From Brasil
- Languages: English 🇬🇧, Portuguese 🇧🇷, when drunk Spanish, lidt dansk 🇩🇰
- Software Engineer at Sealytix; TA at ITU in Intro to AI, Algorithms & Study Lab
- MSc Software Design (2024 - N/A)
- BSc Architecture
- Yoga & reading loads of books (always accept recommendations)



Avery Elise Brunzman

- She/her
- From San Francisco, California
- Second year SD
- Bachelor in Graphic Communication
- Student job in graphic design
- Volunteer barista at Analog
- Love hiking and sauna

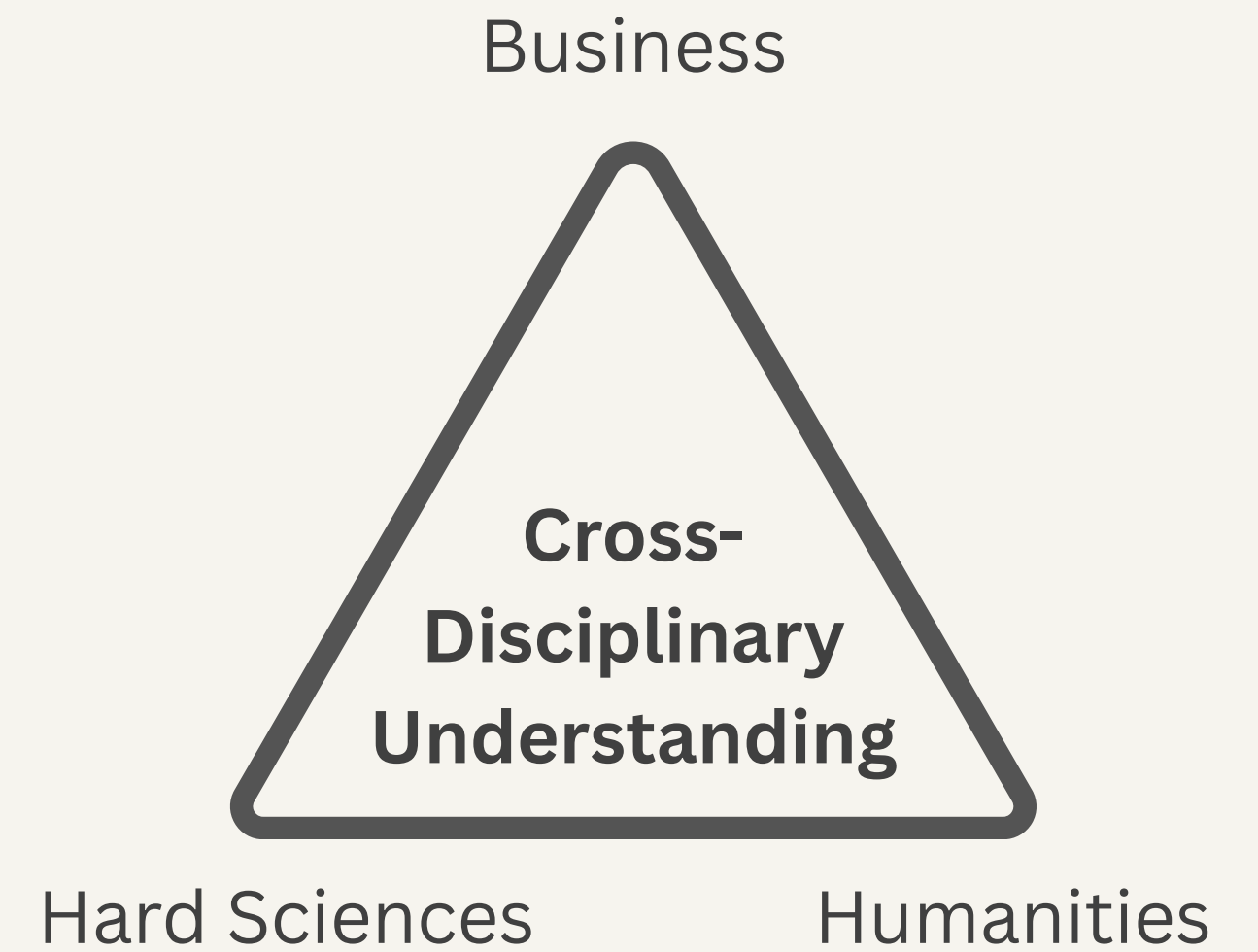


Johannes Alexander Hackl

- He/Him
- From Austria 🇦🇹
- Languages: English 🇬🇧, German 🇦🇹, Portuguese 🇧🇷, very bad Danish 🇩🇰
- Software Developer at Calibras;
TA at ITU in Discrete Mathematics, Algorithms & Study Lab
- MSc Software Design (2024 - N/A)
- BSc Architecture 🏛️
- Footvolley 🏐 & Bouldering 🧗

IT University of Copenhagen

ITU Mission: "... **Create Value with IT in Denmark**"



MSc in Software Design (SD)

1st semester - Autumn

Introductory Programming
15 ECTS

Discrete Mathematics
7.5 ECTS

Software Engineering
7.5 ECTS

2nd semester - Spring

Introduction to Database Systems
7.5 ECTS

Algorithms and Data Structures
7.5 ECTS

Specialisation Course
7.5 ECTS

Elective or Specialisation course
7.5 ECTS

3rd semester - Autumn

Research Project
7.5 ECTS

Specialisation Course
7.5 ECTS

Specialisation Course
7.5 ECTS

Elective or Specialisation course
7.5 ECTS

4th semester - Spring

Thesis
30 ECTS

MSc in Software Design (SD)

- Business Analytics and Data Science
- Creating Digital Systems
- Software Development Technology
 - foundational competences
- Software Development Technology
 - programming competence

Discret
7.5 EC

Frameworks and Architectures for the Web
Functional Programming
Introduction to Artificial Intelligence
Mobile App Development

Specialisation Course
7.5 ECTS

Elective or Specialisation course
7.5 ECTS

3rd semester - Autumn

Research Project
7.5 ECTS

Specialisation Course
7.5 ECTS

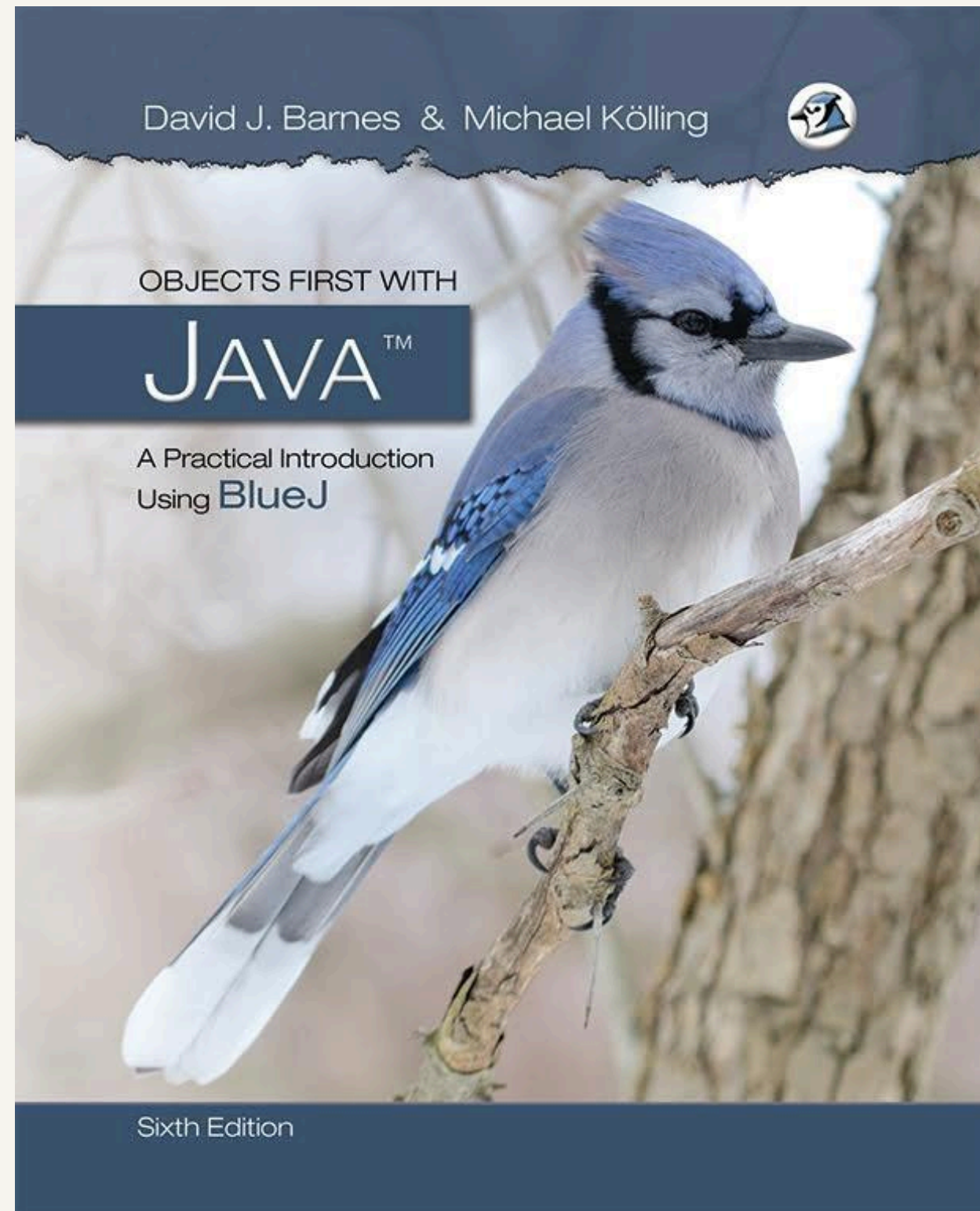
Specia
7.5 EC

Applied Algorithms
Introduction to Security, MSc*
Big Data Management (technical)
Data Mining and Machine Learning**
Distributed Systems
Operating Systems and C
Practical Concurrent and Parallel Programming
Technical Interaction Design

4th semester - Spring

Thesis
30 ECTS

Introductory Programming



This is BlueJ

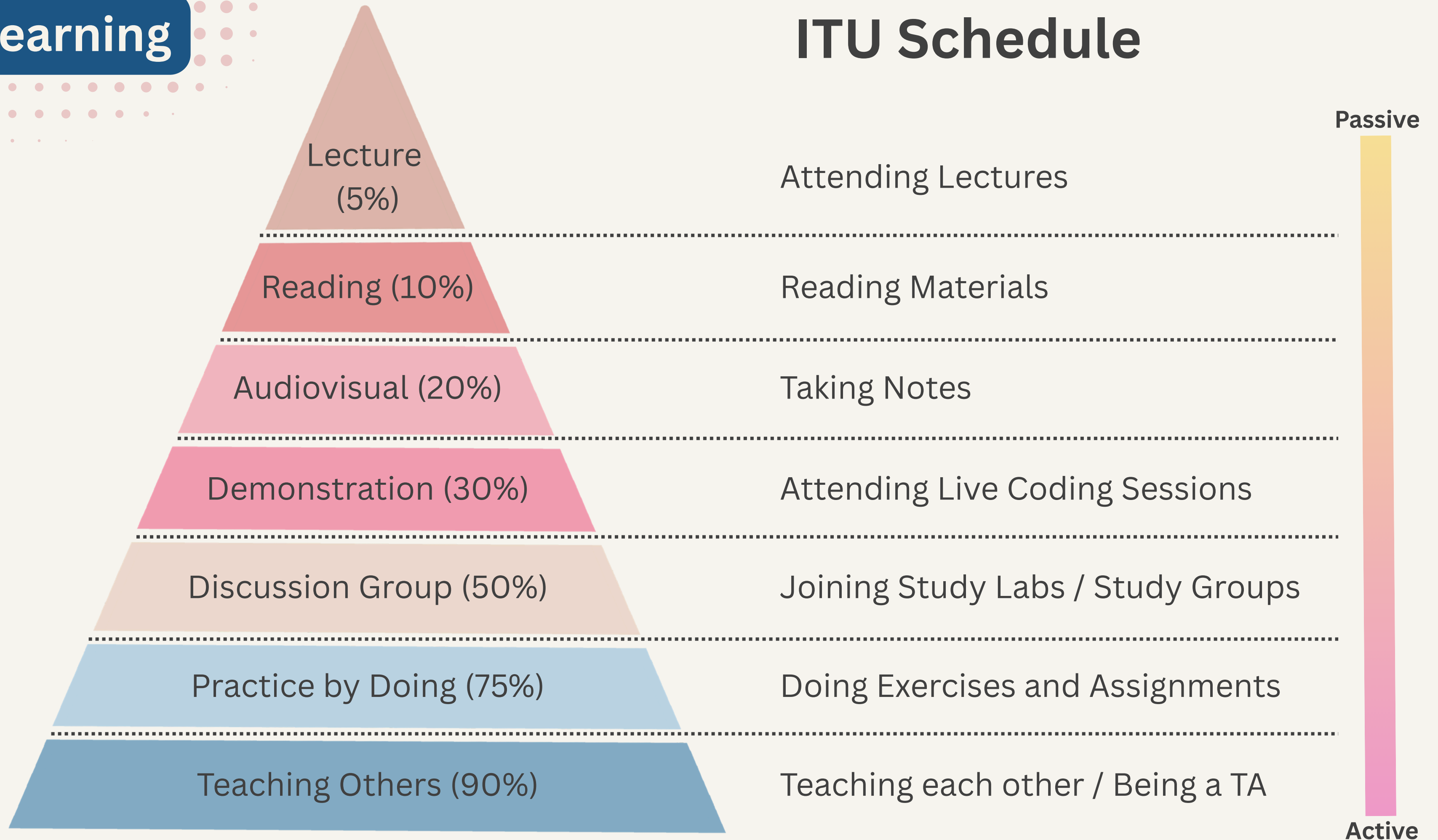


SD: Ignore BlueJ, VS Code Only

They're just different interface for coding (IDE)

Learning

ITU Schedule



Learning with AI

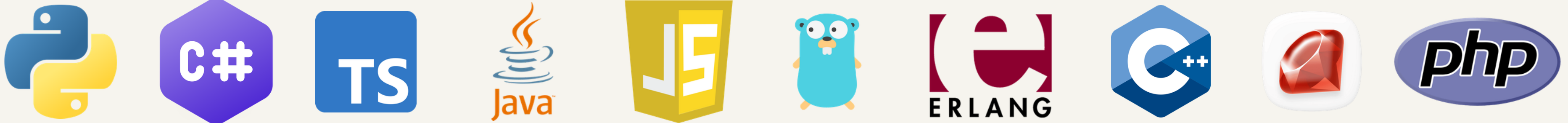
- Constantly ask "why?"
- Make a habit of reading code
- Take ownership and stay curious
- Treat AI as an essential skill
- Be honest with your progress

Vibe coders after sending
AI code to production



Programming Languages

- A **programming language** allows us (humans) to instruct computers what to do for us
- There are **many** (100.000s) of languages
 - Java, JavaScript, C, C++, C#, PHP, ...
- They have **different** characteristics
 - each has different pros & cons
 - the "best" language depends on your goals
 - Java can be used for small & large programs for different computers

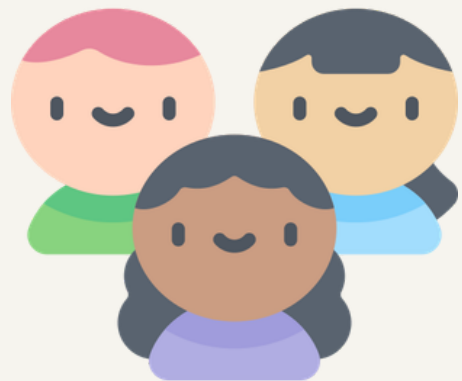


Programming Languages



- Imperative Languages
 - Sequences of instructions
 - C, Pascal, ...
- Object-Oriented Languages
 - Manipulation of objects (abstractions from real world)
 - Java, C#, ...
- Functional Languages
 - "Mathematical" functions
 - ML, F#, ...
- Logical Languages
 - Mathematical Logic
 - Prolog, ...

Layers of Languages and Compiler



Human Language

```
Keep adding to the count
as long as it's under 10.
```

</> Code

High-level Programming

```
while(x < 10){
    x = x + y;
}
```

Translate



Compiler is
the translator

Symbolic Machine Code

```
L: push x
   push y
   add
   pop x
   goto L
```

Machine Code

```
ff8a: push ffa8
      push 8a08
      add
      pop ffa8
      jump ff8a
```

Micro Code

```
push ffa8
noop p_reg %pc
noop push %r1
add store %pc
push 8a08
noop
```

Hardware



Read

```
10000110010101
01110101101001
01000100100100
10101011111111
00001010101010
11010110010010
01010101010110
11010101010110
110111010001 ...
```

Coding Toolbox

1



JDK: Compiles and runs your code.

2



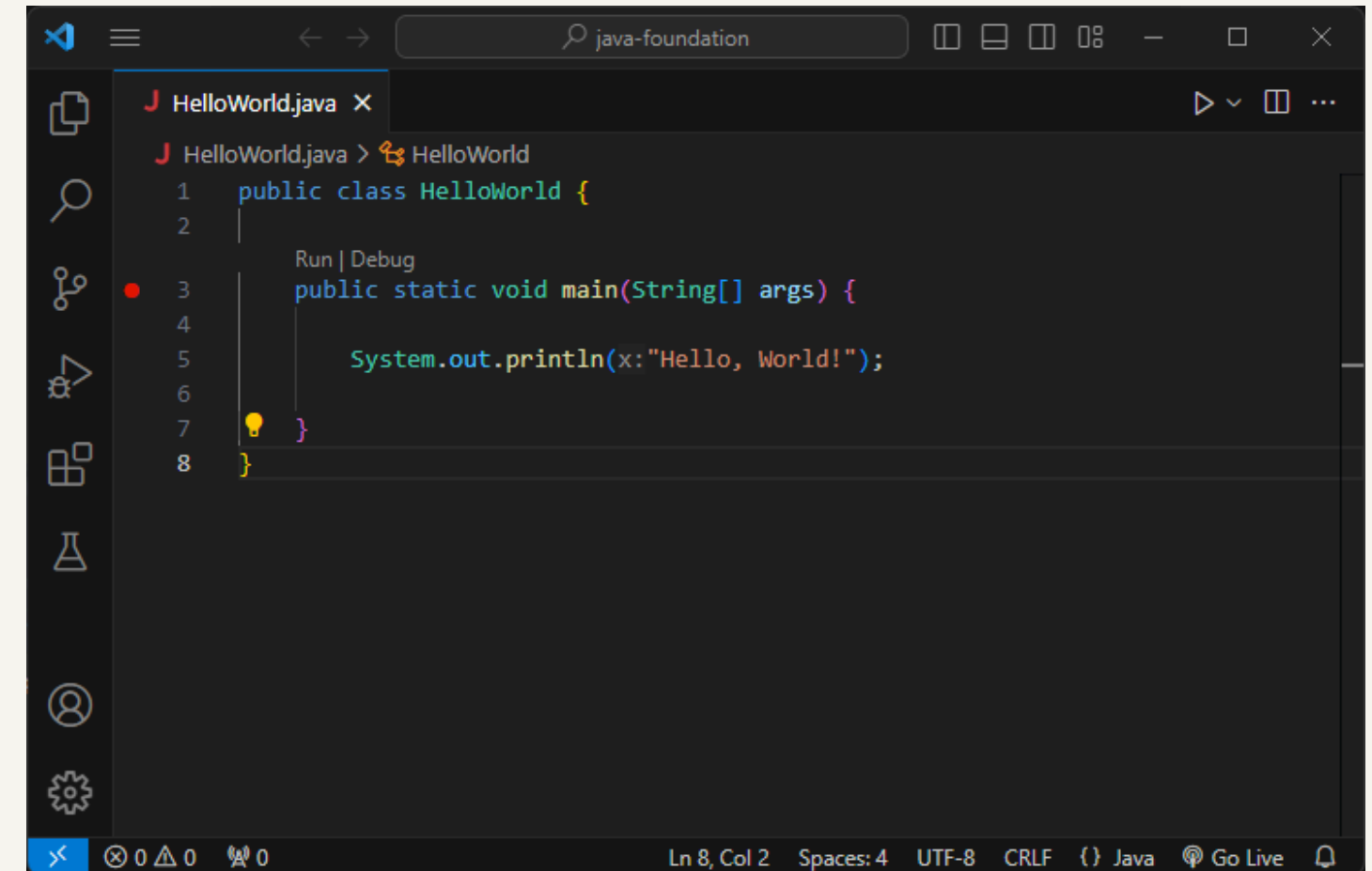
IDE: The code editor you will write in.

3

VS Code Java Extension Pack:
autocomplete and error checking.

Coding Pack for Java

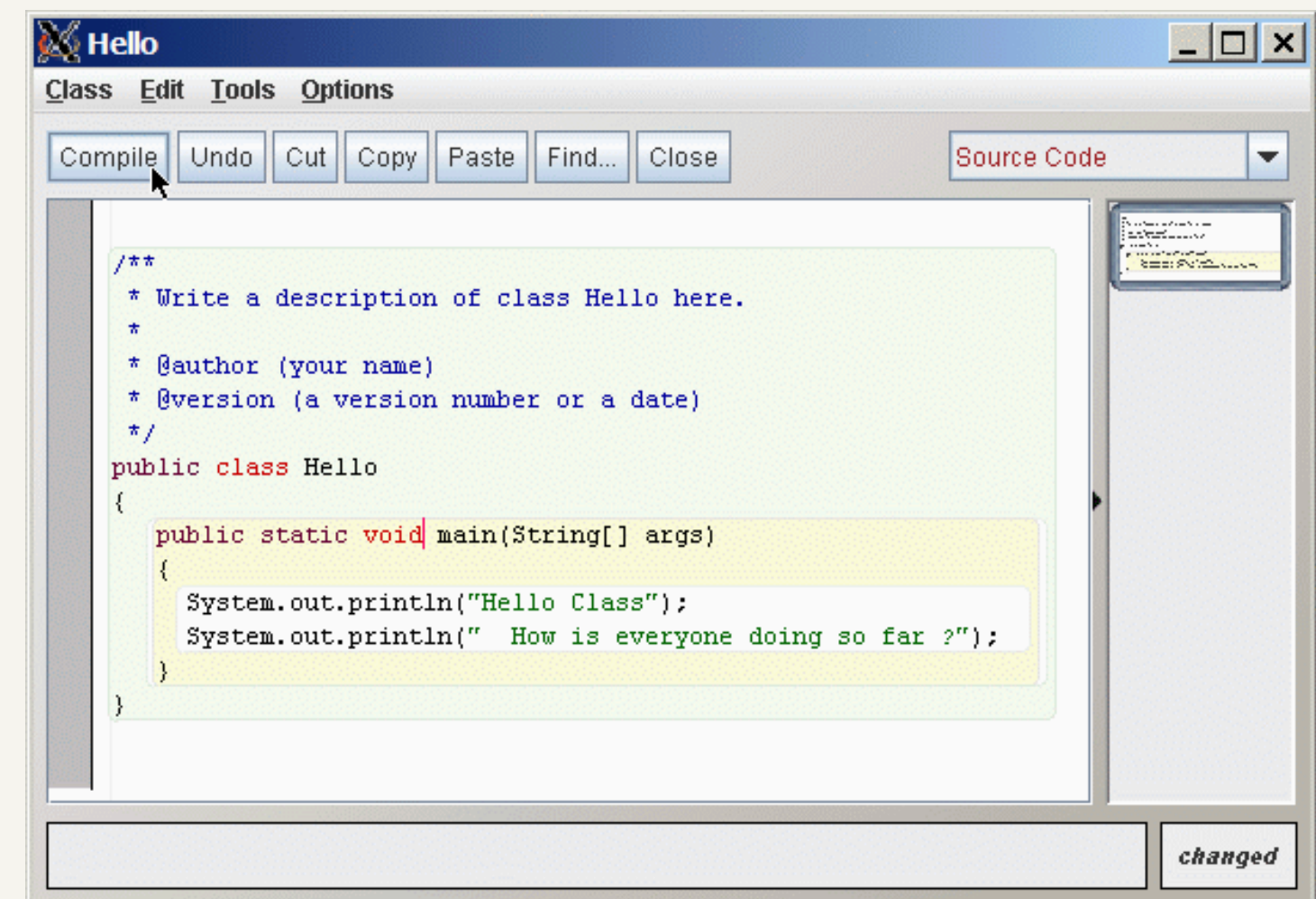
A single installer sets up the [JDK](#), [VS Code](#), and all required extensions.



A screenshot of the Visual Studio Code editor interface. The top bar shows the file name 'HelloWorld.java' and a search bar with 'java-foundation'. The editor window displays the following Java code:

```
1 public class HelloWorld {  
2  
3     Run | Debug  
4     public static void main(String[] args) {  
5  
6         System.out.println(x:"Hello, World!");  
7     }  
8 }
```

The status bar at the bottom indicates 'Ln 8, Col 2', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java'.



A screenshot of a Java IDE window titled 'Hello'. The menu bar includes 'Class', 'Edit', 'Tools', and 'Options'. The toolbar contains buttons for 'Compile', 'Undo', 'Cut', 'Copy', 'Paste', 'Find...', and 'Close'. A dropdown menu is set to 'Source Code'. The main text area contains the following Java code:

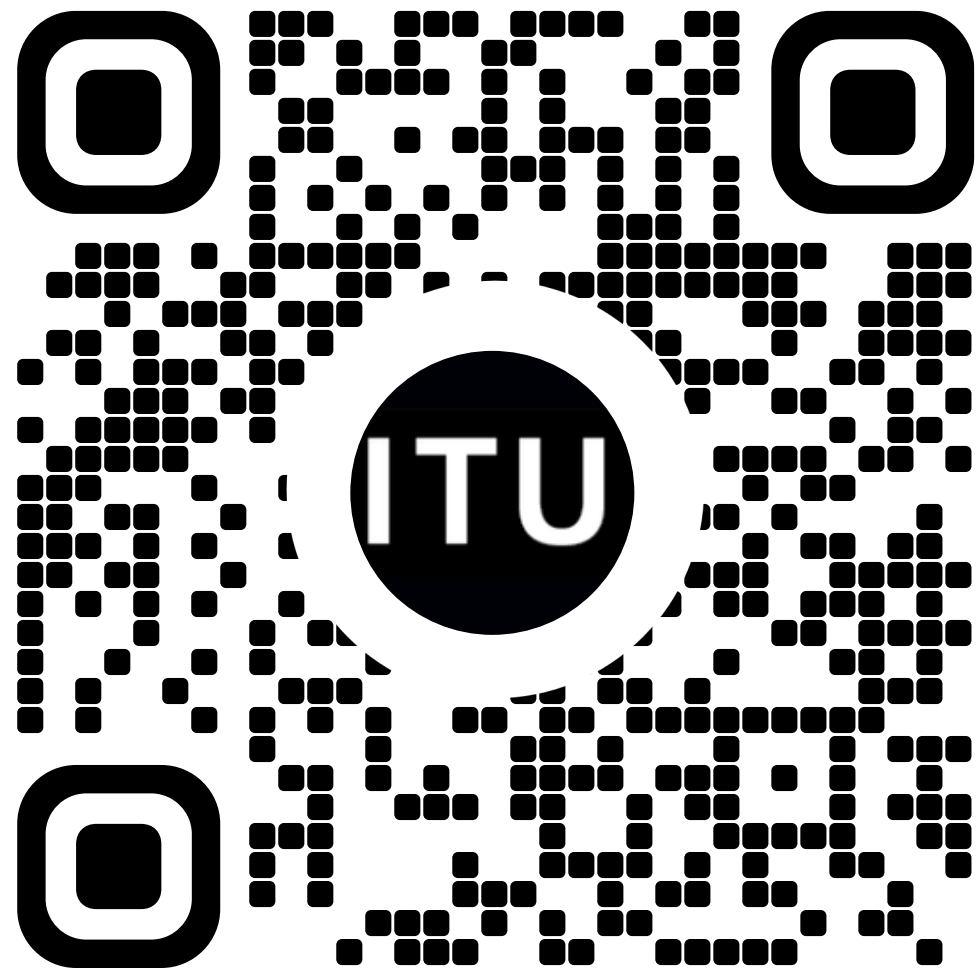
```
/**  
 * Write a description of class Hello here.  
 *  
 * @author (your name)  
 * @version (a version number or a date)  
 */  
public class Hello  
{  
    public static void main(String[] args)  
    {  
        System.out.println("Hello Class");  
        System.out.println(" How is everyone doing so far ?");  
    }  
}
```

The status bar at the bottom right shows 'changed'.

BootCode - Online IDE Built for BootIT

Access the website via a laptop by...

Scanning the QR code



Entering the web address directly

BootCode - ITU BootIT

BootCode is the beginner-friendly Java coding platform for ITU BootIT, the pre-master computer science bootcamp. Write, run, and submit Java exercises directly in your browser.

cobblerscoders.tech

<https://cobblerscoders.tech/>

Clicking the link on the course platform



<https://learnit.itu.dk/course/view.php?id=3026730>

Your First Java Program (Probably!)

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

Your First Java Program (Probably!)

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

- **public class Main** is just the container for your code.
- **public static void main** is the front door of your app—it's where the program starts running.
- Don't worry about them for now. Simply put your code inside the main function's body.

Your First Java Program (Probably!)

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

- **System.out.** A built-in toolbox provided by Java.
- **println()** Prints the text and moves to a new line.
- **print()** Prints the text without starting a new line.

- The actual message you want to print.
- Put it in the "here".

Exercise - Printing a Message

1

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

Print a sentence to say hello to your new university.
For example: Hello ITU!

Run and test your code with BootCode,
then submit when you are confident with your solution.

Different Values

String: “test” between double quotes

```
public static void main(String[] args) {  
    System.out.println("Hello World!");  
}
```

Integer: whole number

```
public static void main(String[] args) {  
    System.out.println(42);  
}
```

Double: decimal number

```
public static void main(String[] args) {  
    System.out.println(3.14);  
}
```

Exercise - Printing Different Values

2

```
System.out.println("Hello World!");  
System.out.println(42);  
System.out.println(3.14);
```

Your new friend is interested to know you better. Here are 3 questions from them, in order:

1. What's your name? — print it as **text**
2. Where do you live, in Danish zip code? — print it as a **whole number**
3. How long have you been here, in decimal years? — print it as a **decimal number**

Print your three answers in that exact order, **each on its own line**.

Note: if you don't feel like answering with your actual info, that's fine

— just print any 1. text, 2. whole number, 3. number with a decimal point, in that order.

Variables

Type

Name

Value

```
int x = 42;  
System.out.println(x);
```



Value: What is actually stored?

Name: Identifier used to refer to this box.

int

Type: Shape of the box; What can be put in?

Types

int

```
int x = 42;  
System.out.println(x);
```

1, 2, 0, -2, -420, 1000000, 2147483647

String

```
String s = "hi";  
System.out.println(s);
```

"hi", "hello world", "@#\$%&*(),.,;", "14"

double

```
double d = 3.14;  
System.out.println(d);
```

1.5, 3.1415, -27.15, 1.0, 6.02E23

boolean

```
boolean b = true;  
System.out.println(b);
```

true and false

Exercise - Printing Different Values Using Variables

3

```
int x = 42;  
System.out.println(x);  
String s = "hi";  
System.out.println(s);
```

```
double d = 3.14;  
System.out.println(d);  
boolean b = true;  
System.out.println(b);
```

Another new friend is asking you the same 3 questions:

1. What's your name? — print it as **String**
2. Where do you live, in Danish zip code? — print it as a **int**
3. How long have you been here, in decimal years? — print it as a **double**

This time, **store each answer in a variable** first, so you can reuse it easily instead of repeating yourself. Then **print the three variables** in that exact order, each on its own line.

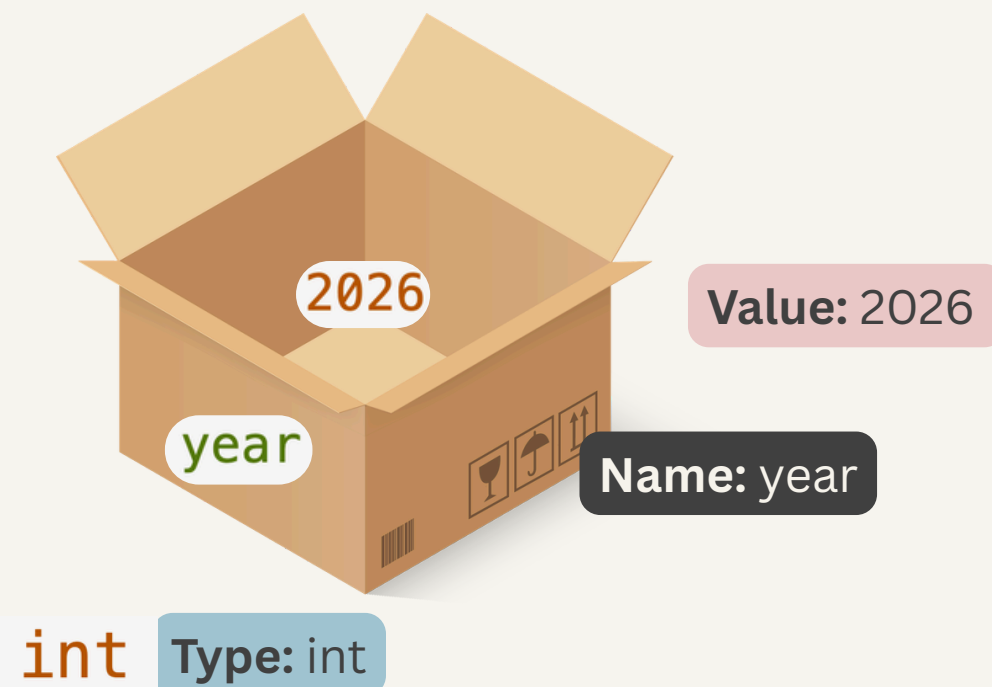
Note: if you don't feel like answering with your actual info, that's fine
— just print any 1. text, 2. whole number, 3. number with a decimal point, in that order.

Variable Assignment

Once a variable is declared and initialized, you can reassign it by using its name to set a new value of the same type.

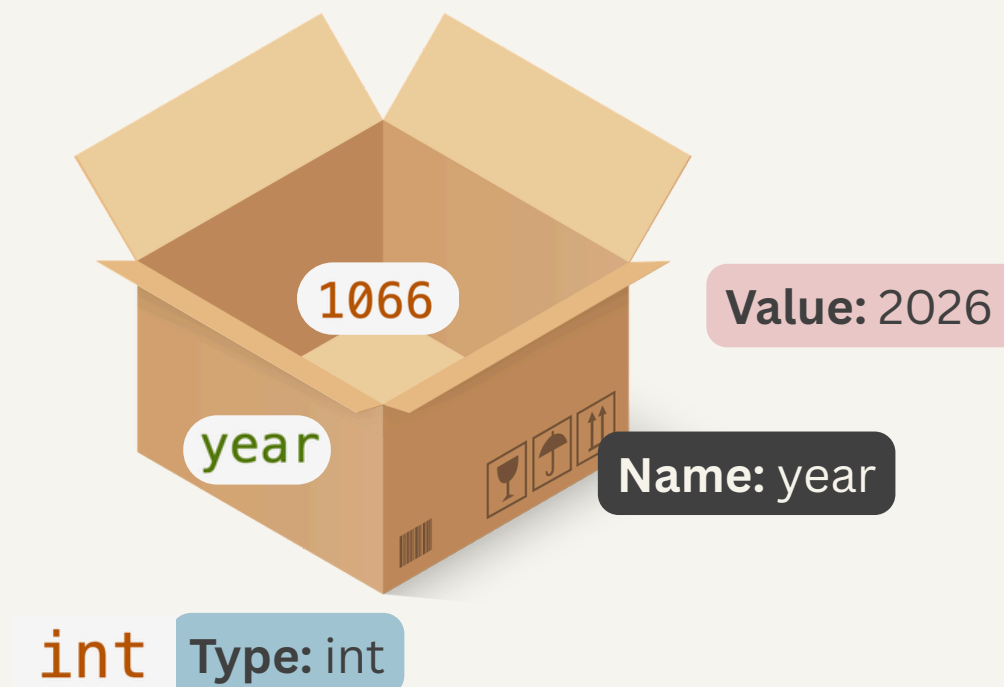
Initial Value ->

```
int year = 2026;  
System.out.print("The year is ");  
System.out.println(year);  
// The year is 2026
```



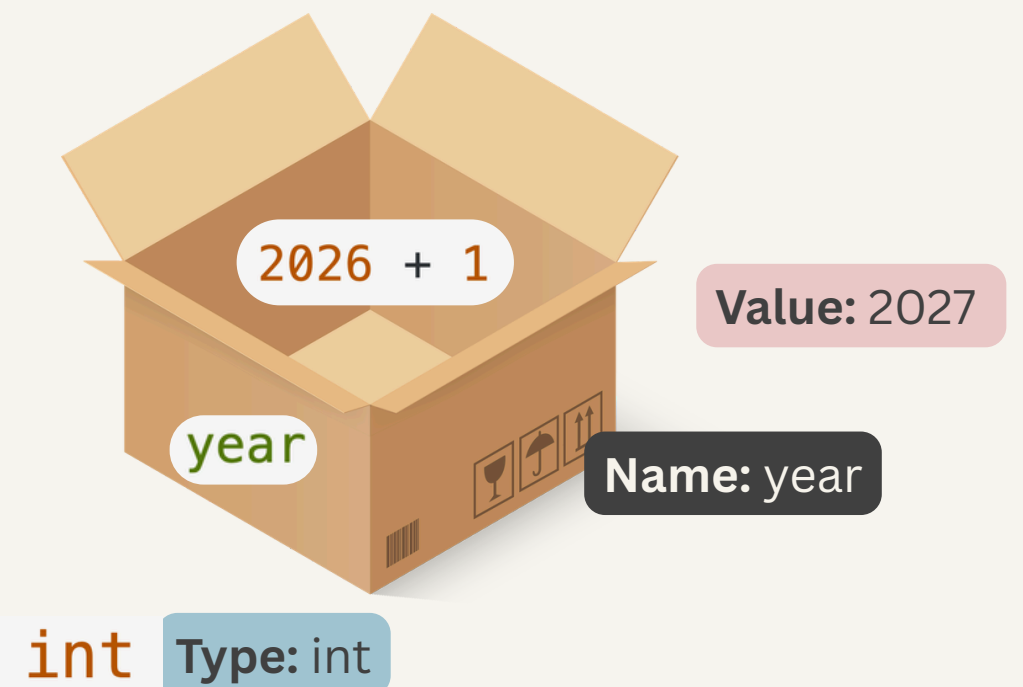
1. Using a new value

```
year = 1066;  
System.out.print("The year is now");  
System.out.println(year);  
// The year is now 1066
```



2. Calculating based on the old value

```
year = year + 1;  
System.out.print("The year is now ");  
System.out.println(year);  
// The year is now 2027
```



Exercise - Assign New Value to a Variable

4

```
int year = 2026;  
System.out.print("The year is ");  
System.out.println(year);  
// The year is 2026  
  
year = year + 1;  
System.out.print("The year is now ");  
System.out.println(year);  
// The year is now 2027
```

Use one **int** variable called **age** (starting at 27) to print "ITU has been open for 27", then print the variable value after.

Then **update** the **SAME variable**, and print "Next year, ITU will have been open for 28".

Fun fact: ITU is the youngest university in Denmark, founded in 1999 — it turns 27 in 2026.

Operations on Numbers

Operators in **int**

```
System.out.println(3 + 3);
```

```
int x = 3;  
System.out.println(x + x);
```

 6

```
x = 2;  
System.out.println(x * x);
```

 4

```
x = 6;  
System.out.println(x / 3);
```

 2

Operators in **double**

```
System.out.println(2.2 + 2.2);
```

```
double x = 2.2;  
System.out.println(x + x);
```

 4.4

```
x = 2.5;  
System.out.println(x * 3);
```

 7.5

```
x = 5.0;  
System.out.println(x / 2);
```

 2.5

Anatomy of Java Code

- **Statement:** A complete unit of execution (usually ending with a semicolon ;).
- **Expression:** A code snippet that evaluates to a single value.
- **Literal Value:** A fixed value written directly in the source code.
- **Variable:** A named storage location of a specific type that holds a value.

Variable Literal Value

```
int x = 2;  
System.out.println((4 + (x * x) + (4 - x)) / x);
```

Expressions

Statement

Exercise - Use Numeric Operators

5

```
System.out.println(3 + 3);    // 6
```

```
int x = 2;  
System.out.println(x * x);    // 4
```

```
int y = 6;  
System.out.println(y / 3);    // 2
```

Fun fact: According to ITU's 2025 figures, about 23.4% of students are international. The Master in Software Design program admits around 130 students per year.

Write code to **calculate and print** the estimated number of international students in this program.

Operations on Strings

+ is String Concatenation Operator

```
String hi = "Hello ";  
String world = "World!";  
String greet = hi + world;  
  
System.out.println(greet);  
// Hello World!
```

It also works between Strings and numbers:

```
int year = 2026;  
System.out.println("The year is " + year);
```

Exercise - Concatenate Strings

6

```
String hi = "Hello ";  
String world = "World!";  
String greet = hi + world;  
  
System.out.println(greet);  
// Hello World!
```

```
int year = 2026;  
System.out.println("The year is " + year);
```

Ask the person sitting next to you for their name, then modify the code to greet them personally: Hello my friend, {Name}!

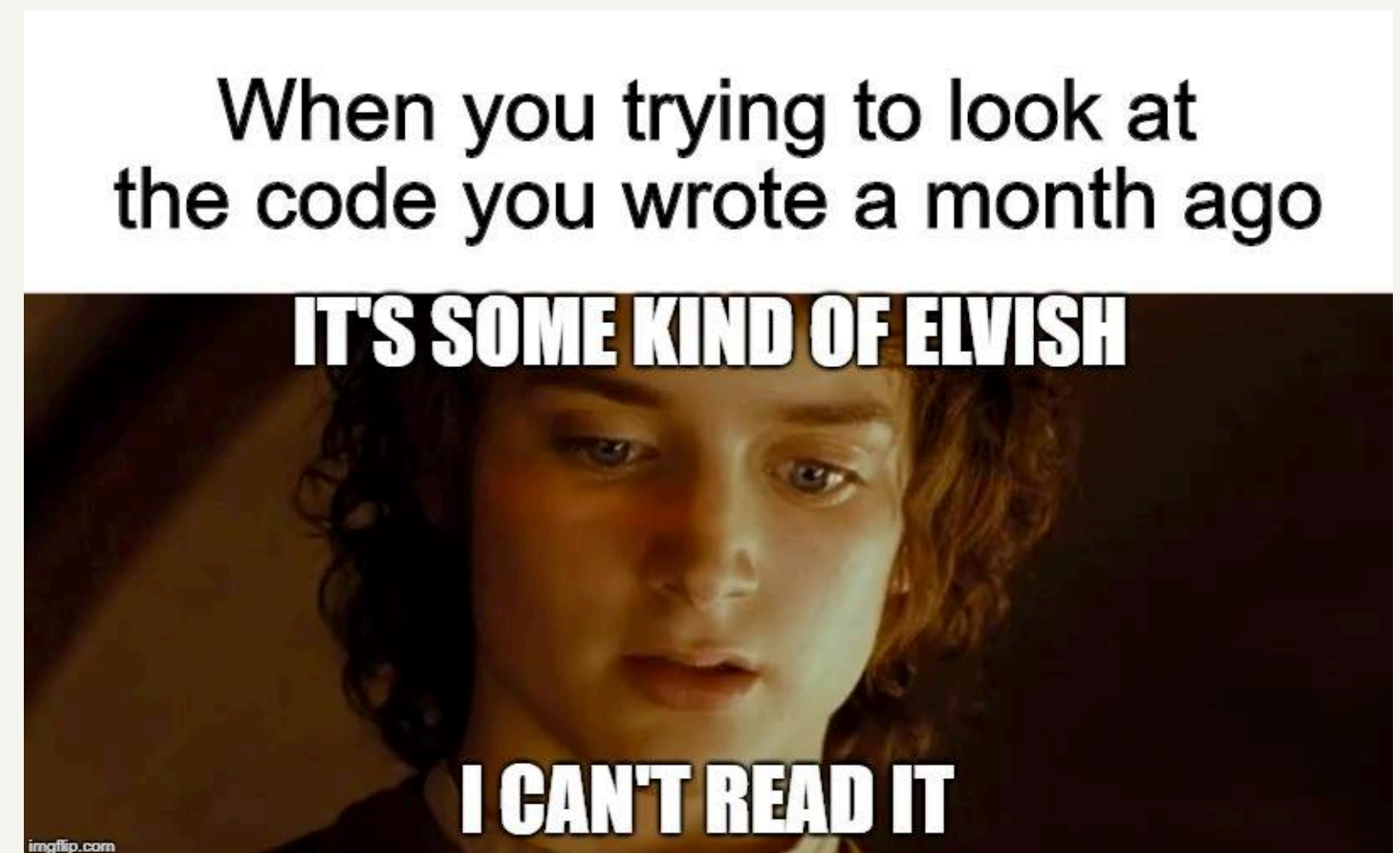
Declare a variable to hold the name, and print the **concatenated sentence**.

Comments

Text in the source code meant for **humans** (and AI) to **read**, which is completely **ignored by** the **compiler** when executed.

Why use them?

- Easier to understand / read the code
- Help others understand the code
- Help future you understand the code



Comments

- Use `//` for single line comment

```
public static void main(String[] args) {  
    // Hi, I am a comment!  
    System.out.println("Hello World!");  
}
```

- Use `/* text in between */` for multiple lines comment

```
public static void main(String[] args) {  
    /* I am in fact also a comment,  
    but I can span multiple lines  
    :-) */  
    System.out.println("Hello World!");  
}
```

Exercise - Currency Exchange

7

During the break you meet a friend at ITU's own café, Cafe Analog.
A coffee costs 20 dkk. Your friend says that is cheap — but you want to see it in euro.
This code converts the other way, from euro to kroner:

```
public class Main {  
    public static void main(String[] args) {  
        int eur = 100;  
        double dkk = eur * 7.45;  
        System.out.println(eur + " euro corresponds to " + dkk + " kr.");  
    }  
}
```

Modify the code so it **converts** the opposite way: **from kroner to euro**.
For a 20 dkk coffee, print: "20 dkk corresponds to {eur} euro."

Note: All the decimals are fine, that is just how doubles print.

Methods

Methods let us **reuse code** and give a snippet a clear **responsibility**.

This does exactly the same as the previous exercise, wrapped in a method:

```
public class Main {  
    static void dkkToEur() {  
        double dkk = 100;  
        double eur = dkk / 7.45;  
        System.out.println(dkk + " kr corresponds to " + eur + " euro");  
    }  
  
    public static void main(String[] args) {  
        dkkToEur();  
    }  
}
```

Exercise - Declare a Method

8

```
public class Main {  
    static void dkkToEur() {  
        double dkk = 100;  
        double eur = dkk / 7.45;  
        System.out.println(dkk + " kr corresponds to " + eur + " euro");  
    }  
  
    public static void main(String[] args) {  
        dkkToEur();  
    }  
}
```

Declare a new **function eurToDkk()** that converts 100 euro to dkk and **prints** "{eur} euro corresponds to {dkk} kr".

Call it from main, after dkkToEur().

Methods with Parameters

But the previous method always converts 100 kr.

With a **parameter**, the same method works for any value:

```
public class Main {  
    static void dkkToEur(double dkk) {  
        double dkk = 100;  
        double eur = dkk / 7.45;  
        System.out.println(dkk + " kr corresponds to " + eur + " euro");  
    }  
  
    public static void main(String[] args) {  
        dkkToEur();  
        dkkToEur(100);  
        dkkToEur(20);  
    }  
}
```

Sequence of Function

When a method is called, the execution jumps to that method. It returns to the calling line after finishes executing .

```
class Valuta {  
2 → static void dkk2eur(double dkk) {  
    double eur = dkk / 7.45;  
    System.out.println(dkk + " kr corresponds to " + eur + " euro");  
}  
1 → public static void main(String[] args) {  
    dkk2eur(100); → 2  
    dkk2eur(20); → 2  
}  
}
```

Within a method, the program executes line by line from top to bottom.

The main method is the entry point of a Java program, where execution always begins.

main → dkk2eur(100) → lines in dkk2eur → dkk2eur(20) → lines in dkk2eur

Exercise - Declare a Method with Parameters

9

```
public class Main {  
    static void dkkToEur(double dkk) {  
        double eur = dkk / 7.45;  
        System.out.println(dkk + " kr corresponds to " + eur + " euro");  
    }  
  
    public static void main(String[] args) {  
        dkkToEur(100);  
        dkkToEur(20);  
    }  
}
```

The previous dkkToEur and eurToDkk always convert a fixed 100.

Rewrite both so they take a parameter instead:

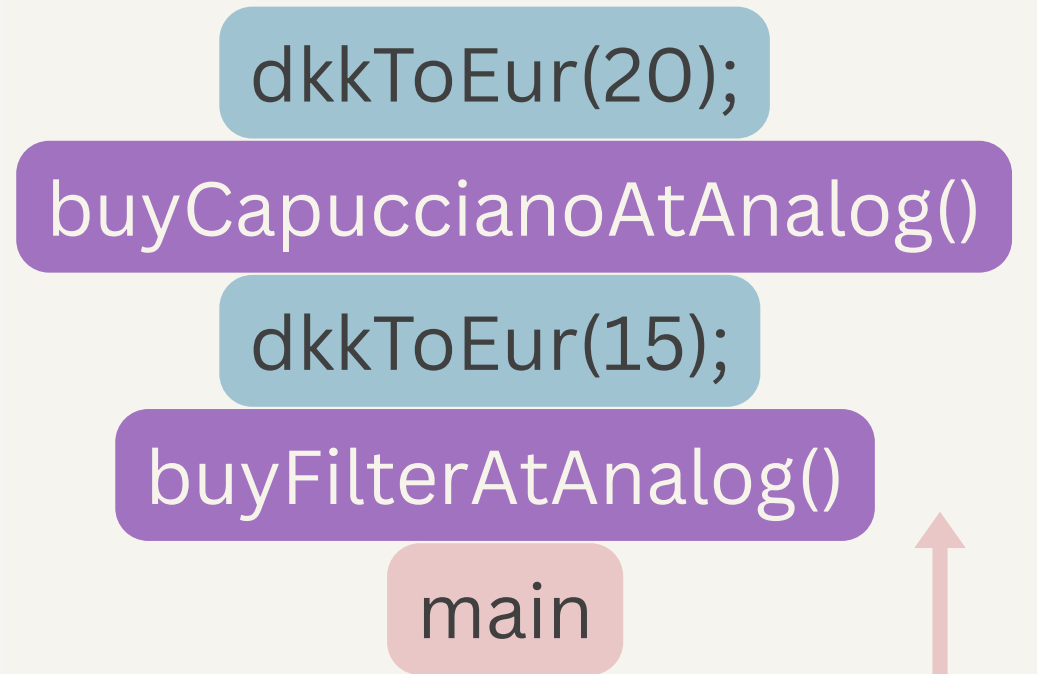
- dkkToEur(double dkk) — prints "{dkk} kr corresponds to {eur} euro"
- eurToDkk(double eur) — prints "{eur} euro corresponds to {dkk} kr"

Call dkkToEur(100) and eurToDkk(100) from main — the output should match the previous exercise, but now the same functions work for any amount

Call Stack

- A method with several parameters can print many different values the same way.
- A method isn't only called from main — it can call another method too.
- A high-level method only knows what to do, without caring how it is done.

```
public class Main {  
    static void dkkToEur(double dkk) {  
        double eur = dkk / 7.45;  
        System.out.println(dkk + " kr corresponds to " + eur + " euro");  
    }  
  
    static void buyFilterAtAnalog() {  
        System.out.println("A cup of filter coffee cost " + 15 + " dkk");  
        dkkToEur(15);  
    }  
  
    static void buyCapuccianoAtAnalog() {  
        System.out.println("A cup of capucciano cost " + 20 + " dkk");  
        dkkToEur(20);  
    }  
  
    public static void main(String[] args) {  
        buyFilterAtAnalog();  
        buyCapuccianoAtAnalog();  
    }  
}
```



Exercise - Methods Call Stack

10

At ITU every course is worth ECTS points, and a full semester adds up to 30 ECTS. You are starting Software Design. **Write `printCourse(String name, double ect, int semester)`**, to print one course's line.

Then **build one layer on top of it**: a method per semester that calls `printCourse` for each of that semester's courses.

- `printFirstSemester()` calls `printCourse()`: Introductory Programming (15 ECTS), Discrete Mathematics (7.5 ECTS), Software Engineering (7.5 ECTS)
- `printSecondSemester()` calls `printCourse()`: Introduction to Database System (7.5 ECTS), Algorithm and Data Structures (7.5 ECTS)

main calls **`printFirstSemester()`** and **`printSecondSemester()`**.